

One Oracle, Every Backend: The CPU Bit-Exactness Law and How It Is Enforced

Hanzo AI — ML Systems

hanzo-kernel Paper 07.4 — part of the *One Source, Every Backend* series

Abstract

The most expensive bugs in a multi-backend inference engine are not crashes; they are silent numeric divergences, where the same model produces coherent text on one GPU and degenerate repetition on another from byte-identical weights. This paper isolates the cause — “the same operation, a different kernel per backend, therefore a different numeric behavior” — and states the law that makes the class structurally impossible: **the CPU runtime is the single bit-exact reference for every backend**. It is not an accelerator target; it is the *oracle*. Every operation ships with a plain-Rust reference whose scalar semantics the device code mirrors line-for-line; because the CPU executes a pointwise operation in program order, its result is the *definition*, and a GPU backend may differ only by floating-point re-association where a reduction reorders a sum — a bounded, explained difference, not a bug to hunt. We show how the law is anchored (a mirror between host and device code), why it holds (index-locality plus in-order execution), and how it is enforced continuously: a nightly three-backend GPU CI on self-hosted NVIDIA, AMD, and Apple machines that runs the same source against the oracle and fails on any divergence. The cross-backend bit-exactness results — including a single attention implementation that is the structural cure for the repetition collapse — are the evidence that the law holds in practice.

1 The pain: coherent on Metal, collapsing on CUDA

A concrete failure names the whole problem. Qwen3-8B-Q4_K_M, a single quantized model file, generates coherent text on Apple Metal — “The capital of France is **Paris**” — and, from the byte-identical file (SHA-256 verified) with the same sampler, *collapses into repeated-token degeneration* on CUDA and Vulkan during long generation. Nothing about the model changed. What changed is the kernel: flash attention’s accumulated numeric behavior on the CUDA/Vulkan tensor-core path differs from the Metal path, and only the former drifts into the repetition attractor. The same operation (attention) has three implementations and therefore three numeric behaviors, and one of them is wrong in a way no unit test on that backend alone would flag as “wrong” — it produces finite, plausible numbers that steer generation into a ditch.

This is the defining bug class of a multi-backend engine, and it has three properties that make it brutal: it is *silent* (no crash, no NaN), it is *backend-specific* (passes on the box you tested), and it is *emergent* (visible only in long generation, not in a single forward). You cannot test four copies of a kernel *into* agreement; agreement has to be a structural property of having one definition.

2 The idea: the CPU is the definition, not a backend

The law is one sentence: **the CPU runtime is the single bit-exact reference every backend is measured against.** The CPU is not “another target that also happens to work.” It is the *oracle* — the place where the operation’s meaning is defined — and every GPU backend is correct exactly insofar as it reproduces the oracle’s bits.

Why can the CPU play this role, and why *bit-exact* rather than merely close? Because a pointwise (Map) operation is *index-local* — output element i depends only on inputs at index i — and the CPU runs it in program order. There is no reordering, no re-association, no fused-multiply-add surprise: the CPU result is the arithmetic the source literally specifies. A GPU backend running the same source computes the same index-local arithmetic and must agree *to the bit*. The only sanctioned source of difference is a **reduction**: a **sum** over a row, which a GPU computes with a tree-reduction across threads and therefore in a different *order* than the CPU’s left fold. Float addition is not associative, so the reordered sum can differ in the last bits. That difference is *bounded* (it is f32 re-association noise, and a small- k control confirms it does not grow with the reduction length — the signature of per-element rounding, not a decode defect) and *explained* (it lives only at reductions, nowhere else). Everything that is index-local is bit-identical; everything that reduces is identical up to a characterized rounding. Nothing is unexplained.

3 The mechanism: a mirror, held by test

The law is anchored by a discipline: for every op, the device code is a *line-for-line mirror* of a plain-Rust reference. The fusion engine’s device morphism table mirrors its host table; every island’s **default** arm *is* the CPU semantics ([4]); every autotune variant is checked against the oracle ([3]). Listing 1 shows the shape — the device `apply_un_dev` and the host `apply_un` are the same expressions — and that identity is *why* a fused kernel equals its reference bit-exactly rather than approximately.

Listing 1: The mirror. The device morphism table (left, a `#[device] fn`) and the host reference (right, plain Rust) are the same arithmetic written twice. Bit-exactness is a consequence of the mirror, not a tolerance that happens to pass.

```
// device ([device]) // host reference (plain Rust)
UnOp::Silu => x * dsigmoid::<F>(x), UnOp::Silu => x / (1.0 + (-x).exp()),
UnOp::Gelu => { let c = F::new(0.797_884_56); UnOp::Gelu => { const C: f32 = 0.797_884_56;
    let inner = c*(x + F::new(0.044715)*x*x*x); 0.5*x*(1.0 + (C*(x + 0.044715*x*x*x)).tanh()) }
    F::new(0.5)*x*(F::new(1.0)+inner.tanh()) }
```

The oracle is enforced across the whole op library — `rms_norm`, `layer_norm`, `rope` (both the NeoX half-split and GPT-J interleaved conventions), `sdpa` (online/flash softmax), the GDN `conv1d/gating/scan`, the quantized matvecs, and the fusion chains — across 36 CPU tests. Each asserts the device result equals the plain-Rust reference within the characterized reduction tolerance (and, for index-local ops, to the bit).

4 Enforcement: a nightly three-backend GPU CI

A law nobody checks is a hope. The oracle is enforced continuously by a GitHub Actions workflow that runs a three-backend matrix on *self-hosted* boxes — one runner label per backend (Listing 2): `cuda` → `spark` (NVIDIA GB10, sm_121), `rocm` → `evo` (AMD Strix Halo, gfx1151, ROCm 7.x),

metal → dbc (Apple M4 Max). Each runner builds the engine for its backend and runs the same bit-exactness tests against the same CPU oracle.

Listing 2: The nightly matrix (`.github/workflows/gpu-tests.yml`). One self-hosted runner per backend; the same tests, the same oracle. Serial execution keeps a single GPU consumer at a time — a necessity, not fastidiousness (§4).

```
on: { workflow_dispatch: {}, schedule: [ { cron: '0 8 * * *' } ] } // nightly + on demand
jobs:
  test:
    strategy: { matrix: { backend: [cuda, rocm, metal] } } // cuda->spark rocm->evo metal->dbc
    runs-on: [self-hosted, "${{ matrix.backend }}"]
    steps:
      - uses: ../.github/actions/gpu-guard // one GPU consumer at a time
      - run: cargo test -p hanzo-ml --features "${{ matrix.backend }}" -- --test-threads 1
```

Two operational details are load-bearing. The workflow runs *nightly and on demand*, not per push, because these boxes double as development and benchmark machines. And it runs `--test-threads 1` behind a GPU-guard step because concurrent GPU initializations on the ROCm box race in the kernel-fusion driver (`kfd`) and wedge it until reboot — so “one GPU consumer at a time” is a hard serialization requirement, enforced by both the guard and the concurrency group. The practical result is that “the same op has the same numbers on every backend” is a *checked invariant*: a divergence introduced on any backend fails the next nightly run against the oracle.

5 Evaluation: the same source, every runtime, bit-exact

Table 1 is the evidence that the law holds where it counts — not merely that the DSL *compiles* for each backend, but that it *dispatches* through each backend’s real runtime and matches the oracle. The ROCm figure is the falsifiable end-to-end proof that the toolchain seam to ROCm 7.13 works: the same `rms_norm` source, lowered through `cubecl-hip` and dispatched on `gfx1151` (device query, module compile, launch, readback), matches the CPU oracle to $\text{max_rel} = 2.43 \times 10^{-7}$ — well under the 2×10^{-3} reduction-tolerance gate, i.e. pure `f32-reorder` noise.

Path	Op(s)	Result
CPU oracle	rms/layer_norm, rope×2, sdpa, gdn, quant, fuse	10/10 bit-exact (36 tests)
DSL → Vulkan (gfx1151)	rms_norm, softmax, matvec, sdpa	bit-exact, real dispatch
DSL → Metal (M4 Max)	rms_norm, layer_norm, rope, sdpa	bit-exact (2.35e−7)
DSL → ROCm (gfx1151)	rms_norm	max_rel = 2.43e−7

Table 1: The same source, dispatched through each backend’s real runtime, matches the CPU oracle. Values are reduction-order `f32` noise, well under the $2e-3$ gate.

The last row of the story returns to §1. Because attention is written *once* in the DSL — one `sdpa` kernel with an online (flash-style) softmax, lowered to every backend — the “flash vs. eager vs. Metal, three numeric behaviors” fork *cannot occur*. There is nothing to diverge. The repetition collapse is not a bug to fix on CUDA and Vulkan; under one implementation it is a state that is unreachable. That is the difference between testing four kernels toward agreement and having one definition: the first is endless, the second is structural.

6 The takeaway

A multi-backend engine must hold one *numerically vetted* implementation per operation, because “the same op, a different kernel per backend” is a correctness hazard first and a performance question second. The CPU oracle is what makes “numerically vetted” concrete: the reference is the CPU’s in-order arithmetic, the device code is its mirror, the only sanctioned difference is characterized reduction-order rounding, and a nightly three-backend CI turns the whole thing from a principle into a gate. Fusion ([2]), autotuning ([3]), and islands ([4]) all rest on this one foundation — and the collapse method ([7]) uses it as the bit-exact half of every verify-then-delete decision.

References

- [1] Hanzo AI. *One Source, Every Backend: The hanzo-kernel DSL*. Paper 07 (umbrella).
- [2] Hanzo AI. *Fusion Is Composition: Kernel Fusion as the Functor Law*. Paper 07.1.
- [3] Hanzo AI. *The Schedule Is a Value: Comptime-Specialized Autotuning*. Paper 07.2.
- [4] Hanzo AI. *Intrinsic Islands: An Oracle-Anchored Escape Hatch*. Paper 07.3.
- [5] Hanzo AI. *One Oracle, Every Backend: The CPU Bit-Exactness Law*. Paper 07.4 (this paper).
- [6] Hanzo AI. *When the Product Won’t Collapse: An int8 Tensor-Core MMQ Negative Result*. Paper 07.5.
- [7] Hanzo AI. *Verify, Then Delete: Collapsing a Kernel Cartesian Product*. Paper 07.6.